

Polynomial Regression Assignment

Kulshresth

IMT2024065

github.com/htserhsluk/ML-Assignment

1 Objective

The task is to train polynomial regression models for two problems, **var1** and **var2**, and generate predictions for their unseen test sets. The implementation evaluates degrees 1–10 for **var1** and 1–20 for **var2**. The target column is y ; every other training column is treated as an input feature.

The central difficulty is that polynomial expansion grows the feature space very quickly. Higher degree gives more expressive nonlinear functions, but it also creates more interactions, multicollinearity, computational cost, and risk of overfitting. The pipeline therefore combines polynomial expansion with feature screening, standardization, regularization, and validation-based model selection.

2 Data Preparation and Validation Strategy

The training data are read as floating-point arrays. The test set is checked to contain the same feature columns as the training set, while y is taken only from the training data. No information from the test targets is used because they are unavailable and the test set is reserved for final inference.

The training data are split into 80% training and 20% validation using a fixed random seed (2026). This makes comparisons reproducible. The validation set is used for all degree, model, and hyperparameter decisions.

The main selection metric is validation mean squared error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

The final configuration is the one with the **lowest validation MSE**. This is appropriate because the task is regression and MSE directly measures squared prediction error. It also penalizes large mistakes more strongly than small ones. Validation R^2 is reported as an additional descriptive metric, but it is not the selection objective. Using the test set for this decision would leak information and could give an overly optimistic estimate of generalization.

3 Polynomial Expansion and Degree Search

For each candidate degree d , `PolynomialFeatures` creates powers and interaction terms up to degree d . The implementation uses `include_bias=False`; the intercept is instead fitted explicitly by the regression model. This avoids adding a constant polynomial column twice.

The degree search is intentionally data-driven:

- **var1**: test all degrees 1, ..., 10.
- **var2**: test all degrees 1, ..., 20.

Ridge is evaluated first for every degree because it is a stable and relatively cheap regularized model. The five degrees with the best Ridge validation MSE are then passed to the more expensive Lasso and Elastic Net searches. The global winner is selected from all evaluated configurations. Thus a degree is chosen because it performs best on held-out training data, not because a particular degree was assumed to be optimal beforehand.

The final run selected the following configurations:

Problem	Degree	Model	Features	Val. MSE	Val. R^2
var1	5	Lasso	461	0.276078	0.972943
var2	10	Ridge	285	0.251014	0.994328

For **var1**, Ridge reached its best value at degree 5 (MSE 0.454036), then became worse at degrees 6–10. Lasso on the same degree reduced the validation MSE further to 0.276078, making degree 5 the overall winner. For **var2**, Ridge

improved strongly up to degree 10, where it reached MSE 0.251014. Higher degrees did not improve it consistently, and the later Lasso/Elastic Net search did not beat the degree-10 Ridge result. Hence the selected degrees are 5 for `var1` and 10 for `var2`.

4 Why Ridge, Lasso, and Elastic Net?

Polynomial terms are often highly correlated. Regularization reduces variance and controls overly large coefficients.

Ridge minimizes

$$\frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2.$$

The L_2 penalty shrinks coefficients toward zero without usually forcing them exactly to zero. It is useful when many correlated polynomial terms may contribute simultaneously. Ridge is therefore also used as the first-pass model for all degrees.

Lasso minimizes

$$\frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1.$$

The L_1 penalty can set coefficients exactly to zero, producing a sparse model. This is attractive in a polynomial feature space because many generated terms may be unnecessary.

Elastic Net combines both penalties:

$$\frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \left(l_1 \|\beta\|_1 + (1 - l_1) \frac{\|\beta\|_2^2}{2} \right).$$

It can provide sparsity like Lasso while retaining the stabilizing effect of Ridge when features are correlated. This is particularly relevant for polynomial expansions, where related powers and interaction terms can be strongly correlated. The three models are therefore complementary: Ridge emphasizes stable shrinkage, Lasso emphasizes sparsity, and Elastic Net provides an intermediate compromise. The validation set decides which behavior works best for each problem.

5 Data-Driven Choice of α and l_1 Ratio

The code does not rely on a manually chosen list such as $\{0.001, 0.01, 0.1, \dots\}$. Instead, the hyperparameter ranges are generated from the current design matrix, so the search adapts to polynomial degree, dimensionality, and data scale.

Ridge α

The selected polynomial matrix is standardized and its Gram matrix

$$G = \frac{X^\top X}{n}$$

is formed. Let $\lambda_{\max}$ be its largest eigenvalue and let p denote the number of selected features. The code constructs

$$\alpha_{\min} = \frac{\lambda_{\max}}{\max(10, 10p)}, \quad \alpha_{\max} = \lambda_{\max} \max(10, 2\sqrt{n}),$$

and samples 12 values geometrically between these bounds. This produces a scale-aware search from weak to strong regularization. A geometric grid is preferred because useful regularization strengths can span several orders of magnitude.

Lasso / Elastic Net α

For standardized X , the code computes

$$\alpha_{\max} = \frac{\max_j |X_j^\top y|}{n},$$

which is the natural Lasso upper scale for the objective used by the model. Twelve values are sampled geometrically from this value down to a sample-size-dependent lower fraction,

$$\frac{\alpha_{\max}}{20\sqrt{n}},$$

with a floor of $10^{-5}\alpha_{\max}$ when required. The same data-derived alpha grid is used for Elastic Net, so its regularization strength is also adapted to the problem rather than arbitrarily fixed.

Elastic Net l_1 ratio

The l_1 ratio controls the mixture of the L_1 and L_2 penalties: 1 is pure Lasso and values closer to 0 are more Ridge-like. Seven candidate ratios are generated from the selected-feature count p using

$$r = \frac{1}{1 + \ln(1 + p)}, \quad \text{low} = \max(0.05, r), \quad \text{high} = \min(0.98, 1 - r/2),$$

followed by an evenly spaced grid between low and high. For each candidate degree, 12 alpha values are paired with 7 ratios, producing 84 Elastic Net configurations. Twelve additional Lasso configurations are tested, giving 96 L1-stage configurations in total. Validation MSE determines the best choice.

6 Feature Scaling

After polynomial expansion and feature screening, **StandardScaler** is fitted on the training portion only and then applied to validation data using the same parameters. The final scaler is similarly fitted on the complete training data before transforming the test data.

Scaling is important because polynomial terms can have dramatically different magnitudes. For example, if an original feature has values around 10, its degree-6 term can be on the order of 10^6 . Without scaling, the regularization penalty would interact with coefficient magnitudes in an undesirable way and optimization could become poorly conditioned. Standardization places features on comparable scales, making the effective Ridge/Lasso/Elastic Net penalties more meaningful and generally making iterative optimization easier.

Crucially, the validation and test sets are never used to fit the scaler. This prevents preprocessing leakage.

7 Random Candidate Selection + F-Statistic Screening

At high polynomial degrees, the full expansion can become too large to use efficiently. The implementation therefore caps the working matrix at **800 selected polynomial terms**. When the expansion exceeds this cap, the following procedure is used:

1. All terms of total degree 1 and 2 are preserved. These low-order terms are the basic linear, quadratic, and pairwise interaction structure and are not discarded simply because higher-degree terms exist.
2. From the remaining higher-degree terms, at most **2500** candidates are sampled uniformly at random with a fixed, degree-dependent seed.
3. Each sampled candidate is scored using a univariate regression F -statistic with the training target.
4. The highest-scoring candidates are retained until the 800-feature cap is reached.

The method is deliberately a hybrid rather than pure random deletion. Random sampling reduces the amount of higher-degree material that must be scored and stored, while the F -statistic prioritizes terms that show individual association with y . This controls memory use, reduces computational cost, reduces the size of the optimization problem, and makes convergence more manageable at high degrees.

Feature selection is performed **only on the training split** during validation. The validation matrix is transformed using the same selected columns. For the final model, selection is repeated using all available training data and then applied to the test matrix with exactly the same selected columns.

8 Computational and Convergence Techniques

Lasso and Elastic Net use cyclic coordinate selection, giving a deterministic optimization order across runs. The search may involve many model configurations, especially for Elastic Net. Independent configurations are therefore fitted concurrently with **ThreadPoolExecutor**, using an automatically chosen number of worker threads capped at 12. BLAS-level native threading is limited to one thread per worker to avoid CPU oversubscription, where multiple software layers would otherwise compete for the same cores.

For Lasso and Elastic Net, the maximum iteration count is generated from the size of the current matrix:

$$\text{max_iter} = \max(5000, c \max(n, p)),$$

where $c = 20$ during the search and $c = 80$ for the final fit. The search uses a looser tolerance (2×10^{-4}) for speed, while the final model uses a tighter tolerance (10^{-5}) for a more accurate fit. These settings make iteration budgets scale with problem size rather than using one fixed number for every degree.

9 Observed Model-Search Results

The validation results support the degree choices quantitatively. For **var1**, Ridge MSE decreased from 8.836504 at degree 1 to 0.454036 at degree 5, then increased at every degree from 6 to 10. The second-stage search improved degree 5 further to 0.276078 with Lasso and $\alpha = 0.00847393$. Thus adding higher polynomial complexity beyond degree 5 was not beneficial on the validation set.

For **var2**, Ridge MSE fell from 33.481634 at degree 1 to its minimum Ridge value of 0.251014 at degree 10. Degrees 11–15 remained close but were slightly worse, while degrees 16–20 degraded markedly. The Lasso/Elastic Net stage was run on the five best Ridge degrees, [10, 12, 11, 13, 14]. Its best result was Elastic Net at degree 14 with MSE 0.260485, which still did not beat the Ridge degree-10 result. Therefore the final configurations are:

Problem	Degree	Model	α	l_1 ratio
var1	5	Lasso	0.00847393	–
var2	10	Ridge	3.01732170	–

10 Final Training and Inference

Once the best validation configuration is known, the corresponding polynomial degree is rebuilt from scratch using the complete training set. The same feature-selection rule and seed are used, followed by fitting a new scaler on all training rows. The selected Ridge, Lasso, or Elastic Net model is then retrained on **all** training targets.

The trained final model is finally applied to the transformed test data. This step is inference: the model parameters are already fixed, and the only operation is to compute $\hat{y}$ for previously unseen test examples. The predictions are written as a single y column in the required files `IMT2024065_pred_var1.csv` and `IMT2024065_pred_var2.csv`.

11 Summary of the Design Choices

Component	Choice / Rationale
Validation	80/20 hold-out, fixed seed 2026; reproducible and cheaper than repeated cross-validation
Primary metric	Lowest validation MSE; directly measures squared regression error
Secondary metric	Validation R^2 for additional interpretation
Degree search	1–10 for var1; 1–20 for var2
First-pass model	Ridge on every degree; stable and comparatively inexpensive
L1-stage degrees	Five best Ridge degrees: var1 [5, 6, 4, 7, 8]; var2 [10, 12, 11, 13, 14]
Ridge search	12 data-scaled geometric alpha values per degree
Lasso / EN alpha	12 values based on data-derived $\alpha_{\max}$
EN ratios	7 feature-count-dependent l_1 ratios; 84 EN + 12 Lasso = 96 configurations per selected degree
Scaling	Standardize selected polynomial terms before regularization
Feature screening	Cap at 800 terms; preserve all degree ≤ 2 terms, sample up to 2500 higher-degree candidates, then F-screen
Parallelism	ThreadPoolExecutor with up to 12 workers; BLAS native threads limited to avoid oversubscription
Convergence	Search L1 budget scales as $20 \max(n, p)$ with tolerance 2×10^{-4} ; final L1 budget uses factor 80 and tolerance 10^{-5}
Final fit	Refit the selected configuration on all training data
Test use	Final inference only; never used for model selection
Outputs	Exactly two prediction CSV files required by the assignment

12 Conclusion

The approach balances model flexibility and generalization rather than simply increasing polynomial degree. Polynomial expansion supplies nonlinear capacity, feature screening keeps the high-degree design computationally tractable, standardization makes regularization numerically meaningful, and Ridge, Lasso, and Elastic Net provide different forms of coefficient control. Data-driven hyperparameter ranges avoid arbitrary scales, while validation MSE gives one consistent rule for selecting degree, model, and regularization strength.

The final model for each problem is the configuration that achieves the lowest validation MSE among the permitted searches, after which it is retrained on all available training data and used only for final test-set inference.